

PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ
FACULTAD DE CIENCIAS E INGENIERÍA

ALGORITMIA Y ESTRUCTURA DE DATOS

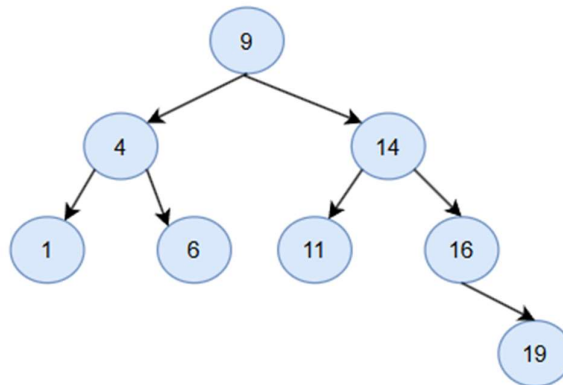
5ta. práctica (tipo B)
(Primer Semestre 2026)

Duración: 1h 50 min.

- **No puede utilizar apuntes, solo hojas sueltas en blanco.**
- En cada función el alumno deberá incluir, a modo de comentario, la forma de solución que utiliza para resolver el problema. De no incluirse dicho comentario, el alumno perderá el derecho a reclamo en esa pregunta.
- No puede emplear plantillas o funciones no vistas en los cursos de programación de la especialidad.
- Los programas deben ser desarrollados en el lenguaje C++. Si la implementación es diferente a la estrategia indicada o no la incluye, la pregunta no será corregida.
- Un programa que no muestre resultados coherentes y/o útiles será corregido sobre el 50% del puntaje asignado a dicha pregunta.
- **Solo está permitido acceder a la plataforma de PAIDEIA, cualquier tipo de navegación, búsqueda o uso de herramientas de comunicación se considera plagio por tal motivo se anulará la evaluación y se procederá con las medidas disciplinarias dispuestas por la FCI.**
- Para esta evaluación solo se permite el uso de las librerías `iostream`, `iomanip`, `limits`, `string`, `ctime`, `cstdlib`, `cmath`, `vector` o `fstream`. Recuerde que no puede emplear variables globales.
- Su trabajo deberá ser subido a PAIDEIA.
- **Es obligatorio usar como IDE CLion. No está permitido el uso de variables globales.**
- **Los resultados que se muestran en los ejemplos son solo referenciales, que su programa funcione para estos datos no garantiza que su programa tendrá la calificación completa. Es responsabilidad del alumno que su desarrollo funcione para diferentes casos.**
- Los archivos deben llevar como nombre su código de la siguiente forma `codigo_LAB5_P#` (donde # representa el número de la pregunta a resolver)

Pregunta 1 (10 puntos)

Durante el curso, hemos visto distintas operaciones que podemos realizar a un Árbol Binario de Búsqueda (ABB) como son insertar nodo, eliminar nodo, etc. En esta oportunidad vamos a implementar una nueva operación cuya lógica debe hacer uso exclusivo de **estructura(s) iterativa(s)**. Con el fin de validar el correcto funcionamiento de su solución, utilizaremos el ABB mostrado debajo:



- a. (9 ptos.) Zig-Zag order: Es un tipo de recorrido en el cual el primer nivel se lee de izquierda a derecha, el segundo de derecha a izquierda, el tercero de izquierda a derecha, y así

sucesivamente. Se pide definir una función de nombre `zigZagOrder` la cual recibe un `ArbolBinarioBusqueda` y genera el siguiente resultado:

Nivel 1: 9

Nivel 2: 14 4

Nivel 3: 1 6 11 16

Nivel 4: 19

Puede usar a lo mucho 2 TAD de apoyo en su solución. Complejidad $O(N)$.

- b. (1 pt.) Implemente en la función `main` el código necesario para que genere el output mostrado.

Nota: Solo se solicita que el desarrollo sea iterativo para el recorrido de la parte a.

Pregunta 2 (10 puntos)

Implementar un programa que lea un archivo de texto, contabilice la frecuencia de cada palabra que contiene, y organice dichas palabras en **26 Árboles Binarios de Búsqueda (ABB)** independientes, uno por cada letra del abecedario español (a a z). Finalmente, el programa debe generar un reporte que muestre para cada letra del abecedario las palabras encontradas, ordenadas alfabéticamente, junto con su frecuencia. Para el desarrollo de la pregunta en el manejo de cadenas puede hacer uso de la STL `string` o trabajar con `char *`

Estructura `Letra`

Debes definir una estructura que asocie una letra del abecedario con su ABB correspondiente.

```
struct Letra{
    char letra;
    struct ArbolBinarioBusqueda arbol;
};
```

Para manejar el abecedario puede hacer uso de la STL `vector` o de un arreglo de estructuras dinámico.

Inicialización

Inicialice el abecedario con las letras y sus respectivos árboles.

Para desarrollar el elemento de ABB use lo siguiente:

```
struct ElementoArbolBinarioBusqueda {
    string palabra;
    int frecuencia;
};
```

Cuando se inserta una palabra que ya existe en el árbol, la operación `insertar` no crea un nodo duplicado: en su lugar, incrementa en 1 el campo `frecuencia` del nodo existente.

Lectura del archivo y llenado de los árboles

Abrir el archivo de texto indicado. Si no puede abrirse, mostrar un mensaje de error y finalizar el programa.

Leer el archivo palabra por palabra (separadas por espacios en blanco) hasta llegar al final del archivo.

Por cada palabra leída, insertarla en el árbol correspondiente según su primera letra. No se debe distinguir mayúsculas de minúsculas al determinar la letra de una palabra (asumir que el archivo de entrada ya está

en minúsculas). Asimismo, se asume que todas las palabras comienzan con una letra del alfabeto español y que el archivo de entrada no contiene signos de puntuación.

Archivo: volveran_las_oscuras_golondrinas.txt

```
volveran las oscuras golondrinas
en tu balcon sus nidos a colgar
y otra vez con el ala a sus cristales
jugando llamaran
pero aquellas que el vuelo refrenaban
tu hermosura y mi dicha a contemplar
....
```

Poema "Volverán las oscuras golondrinas" de Gustavo Adolfo Bécquer (Rima LIII), adaptado sin acentos ni signos de puntuación para fines de este ejercicio.

Inserción por letra

Debe determinar a qué letra del abecedario corresponde la palabra (según su primer carácter), ubicar el árbol correspondiente e insertar el elemento del árbol formado por la palabra y la frecuencia correspondiente.

Búsqueda de letra en el abecedario

La función de búsqueda debe ser eficiente pudiendo ser **$\log(n)$ ó $O(1)$** , se considera descuento si utiliza $O(n)$.

Reporte de frecuencias

Debe recorrer el **abecedario** y, para cada letra, imprimir:

- La letra correspondiente.
- Mostrar en orden cada palabra junto con su frecuencia. Si un árbol está vacío, únicamente se imprime la letra correspondiente.

Restricciones y consideraciones

- Debe utilizar el TAD **ArbolBinarioBusqueda** ya implementado en clase.
- El programa debe manejar correctamente el caso en que el archivo no exista o no pueda abrirse.
- Cada árbol del abecedario es independiente: una palabra solo debe existir en el árbol correspondiente a su primera letra.

Caso de prueba (Volverán las oscuras golondrinas)

=====

REPORTE DE FRECUENCIAS DE PALABRAS EN EL POEMA

=====

a:

a		7
abriran		1
absorto		1
adora		1
ala		1
altar		1
amor		1
ante		1
aprendieron		1
aquellas		3
ardientes		1
asi		1
aun		1

b:

balcon		1
--------	--	---

c:

caer		1
colgar		1
como		3
con		1
contemplar		1
corazon		1
cristales		1
cujadas		1

Continúa...

Al finalizar el laboratorio, comprima la carpeta de su proyecto empleando el programa Zip que viene por defecto en el Windows, **no se aceptarán los trabajos compactados con otros programas como RAR, WinRAR, 7zip o similares**. Luego súbalo a la tarea programa en Paideia para este laboratorio.

Profesores del curso:

Ana Roncal
Fernando Huamán
David Allasi

Rony Cueva
Erasmus Gomez
José Luis Corcuera

San Miguel, 04 de julio del 2026